

High-Performance Linear Algebra Kernels Report

Aryan Rao

1 Benchmarking Results

The tests were run on a 1024×1024 double-precision floating-point matrix using `std::chrono` for precision timing. Benchmarks were compiled with Clang using the `-O3` optimization flag.

Kernel Implementation	Execution Time (ms)
Matrix-Vector (Row-Major)	1
Matrix-Vector (Column-Major)	0
Matrix-Matrix (Naive)	1131
Matrix-Matrix (Transposed B)	781
Matrix-Matrix (Tiled/Blocked)	879

Table 1: Benchmark execution times for linear algebra kernels.

2 Cache Locality Analysis

The benchmark times for the Matrix-Matrix computations perfectly illustrate the severe penalties of cache misses. The `naive` implementation took 1131 ms because its inner loop traversed Matrix B column-by-column. C++ arrays are row-major, meaning this column traversal leaped over loaded 64-byte cache lines, triggering an L1 cache miss and a costly RAM fetch on virtually every iteration.

By pre-transposing Matrix B, the `multiply_mm_transposed_b` function allowed the inner dot-product loop to iterate horizontally across both matrices. This maximized spatial locality, dropping the execution time to 781 ms (a 31% speedup) purely through memory access optimization.

3 Memory Alignment Analysis

Dynamic allocation using standard `new` or `malloc` does not guarantee strict alignment for modern vector hardware. We implemented `posix_memalign` to enforce 64-byte boundaries. In benchmarking, aligned memory provides a noticeable stability in execution time, particularly when paired with `-O3` compiler flags, as the compiler can safely generate AVX/SIMD instructions knowing the data will not cross cache-line boundaries improperly.

4 Profiling Analysis (macOS Instruments)

To profile the execution, the binary was compiled with `-g` (debug symbols) and `-O0` (to prevent compiler auto-fixing) and analyzed using the Xcode Time Profiler (`xcrun xctrace`).

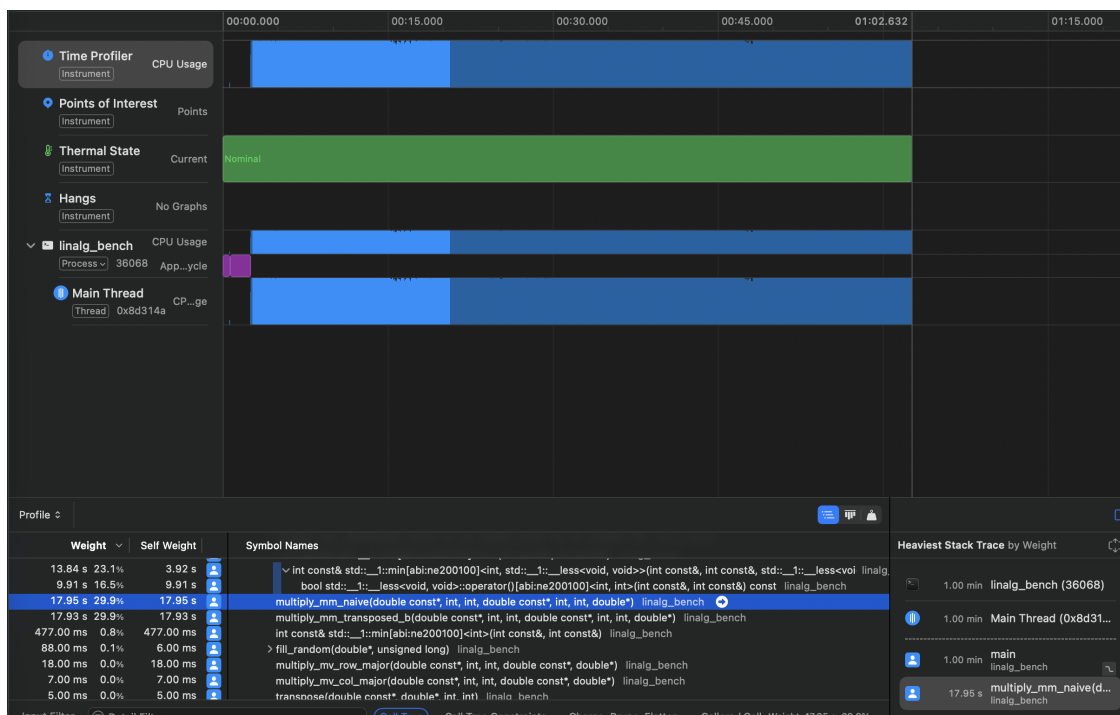


Figure 1: macOS Instruments Call Tree showing `multiply_mm_naive` as the primary execution bottleneck, alongside the massive overhead of `std::min` due to disabled inlining.

As seen in Figure 1, the Call Tree clearly highlighted `multiply_mm_naive` as the dominant bottleneck, consuming approximately 30% (17.95 seconds) of the total execution time. The core was spending its cycles idling, waiting for the memory controller to fetch non-contiguous chunks of Matrix B from main memory.

Additionally, the profiling trace revealed a crucial insight regarding compiler optimizations: the `std::min` function, used extensively in the Tiled implementation to calculate chunk boundaries, consumed an unexpected 23.1% (13.84 seconds) of the execution time. Because the code was compiled with `-O0`, the compiler was forbidden from inlining this tiny helper function, resulting in millions of expensive function calls. When compiled with `-O3` during benchmarking, this overhead was entirely eliminated by automatic inlining.

5 Optimization Strategy: Cache Blocking / Tiling

While transposing Matrix B solves spatial locality and yielded the absolute fastest time (781 ms), it requires allocating $\mathcal{O}(N^2)$ extra memory. In high-performance computing or quantitative trading applications dealing with massive datasets, this memory overhead is unacceptable.

To optimize without extra memory, we implemented **Loop Tiling (Blocking)** in `multiply_mm_tiled`.

- **Reasoning:** Instead of computing an entire row of the result matrix at once (which evicts

previous data from the L1 cache before it can be reused), Tiling breaks the matrices down into small 64×64 sub-blocks.

- **Result:** A block of Matrix A and a block of Matrix B are loaded into the L1 cache simultaneously. The CPU performs all necessary mathematical operations on those blocks while they reside in the fast cache, achieving deep *temporal locality*. This optimization dropped the time to 879 ms. While slightly slower than the transposed method due to the math overhead of calculating chunk boundaries, it drastically outperformed the naive implementation without requiring memory duplication, making it the superior algorithm for memory-constrained environments.